

5. Operators in C

Operators are special symbols that perform operations on variables and values.

Types of Operators

- **Arithmetic Operators:** + , - , * , / , %
- **Relational Operators:** == , != , > , < , >= , <=
- **Logical Operators:** && (and) , || (or) , ! (not)
- **Assignment Operators:** = , += , -= , *= , /= , %=
- **Increment / Decrement:** ++ , --
- **Bitwise Operators:** & , | , ^ , ~ , << , >>

Example:

```
int a = 10, b = 3;
printf("a + b = %d\n", a + b);    // 13
printf("a > b is %d\n", a > b);    // 1 (true)
printf("a && b is %d\n", (a && b)); // 1 (true)
```

6. Control Statements

Control statements decide the flow of execution in a program.

6.1. if Statement

Used to execute a block of code if a condition is true.

```
int n = 10;
if (n > 0) {
    printf("Positive number\n");
}
```

If n > 0 is true,
then the block
inside if runs.

6.2. if...else Statement

Used to choose between two alternatives.

```
int n = -5;
if (n >= 0) {
    printf("Non-negative number\n");
}
else {
    printf("Negative number\n");
}
```

If condition is false,
else block
executes.

3. Basic Structure of a C Program

Every C program follows a basic structure.

```
#include <stdio.h>           // Preprocessor directive
int main()                   // Main function
{
    printf("Hello, C Programming!\n"); // Statement
    return 0;                 // Successful termination
}
```

- `#include <stdio.h>` : Includes the standard input-output header file.
- `int main()` : The `main()` function is the starting point of every C program.
- `printf()` : Used to display output on the screen.
- `return 0;` : Indicates successful completion of the program.

4. Variables and Data Types

Variables

A variable is a named location in memory used to store data.

```
int age = 20;           // Integer variable
float marks = 85.5;     // Floating point variable
char grade = 'A';       // Character variable
```

Data Types in C

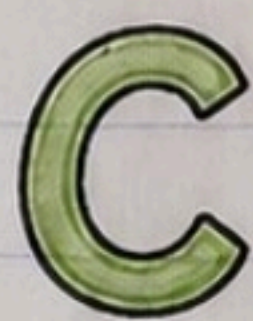
C has several data types to store different kinds of data.

Type	Size (bytes)	Description	Example
int	4	Integer numbers	int a = 10;
float	4	Single precision floating point	float b = 3.14;
double	8	Double precision floating point	double c = 3.14159;
char	1	Character	char ch = 'Z';
void	—	No value / no type	void fun();

- Others: short, long, unsigned int, long long, etc.

Choose the right data type to save memory and improve performance.





C Programming Basic Notes for Beginners - Part-1

1. C Overview

- What is C?

C is a general-purpose, procedural programming language developed by Dennis Ritchie in 1972 at Bell Labs.

It is known for its simplicity, efficiency, and flexibility. C is widely used in system programming, embedded systems, and application development.

- Why Learn C?

- **Foundation:** C is the base for many modern languages like C++, Java, Python, etc.

- **Efficiency:** C programs are fast and memory-efficient.

- **Portability:** C code can be compiled and run on different platforms with little or no modification.

- Features of C:

- Simple and structured

- Middle-level language (combines features of high-level and low-level languages)

- Modular (supports functions)

- Portable

- Rich set of operators and data types

- Applications of C:

- Operating Systems, Compilers, Embedded Systems, Databases, Games, and much more.

2. History of C

C was developed in the early 1970s by Dennis Ritchie at Bell Labs. It was originally used to develop the UNIX operating system. Since then, it has become one of the most popular and influential programming languages.

Keep Coding, Keep Learning!

7. Loops in C

Loops are used to execute a block of code multiple times.

Types of Loops

- **for Loop**: When the number of iterations is known.
- **while Loop**: When the number of iterations is unknown.
- **do...while Loop**: Similar to while loop, but executes at least once.

7.1 for Loop

Used when the number of iterations is known in advance.

```
for (initialization; condition; update) {  
    // code to be executed  
}
```

Example:

```
for (int i = 1; i <= 5; i++) {  
    printf("%d ", i);  
}
```

Output: 1 2 3 4 5

7.2 while Loop

Repeats a block of code while the condition is true.

```
while (condition) {  
    // code to be executed  
}
```

Example:

```
int i = 1;  
while (i <= 5) {  
    printf("%d ", i);  
    i++;  
}
```

Output: 1 2 3 4 5

7.3 do...while Loop

Executes the block once, then repeats while condition is true.

```
do {  
    // code to be executed  
} while (condition);
```

Example:

```
int i = 1;  
do {  
    printf("%d ", i);  
    i++;  
} while (i <= 5);
```

Output: 1 2 3 4 5

8. Decision Making Statements

Decision making statements allow a program to make decisions based on conditions.

Statement	Description	Syntax
if	Executes a block if condition is true.	if (condition) { ... }
if...else	Executes one block if true, another if false.	if (cond) { ... } else { ... }
else if ladder	Checks multiple conditions.	if (...) { ... } else if (...) { ... } else { ... }
switch	Selects one of many blocks.	switch (expr) { case ...: ... }



Understanding loops and decision making is key to solve any problem!

@coders.world

9. Input and Output Functions in C

C provides a rich set of functions to take input and display output.

Types of I/O Functions

- Formatted I/O Functions:
 - `printf()` - Formatted output
 - `scanf()` - Formatted input
- Character I/O Functions:
 - `getchar()` - Read a character
 - `putchar()` - Write a character
- String I/O Functions:
 - `gets()` - Read a string (unsafe)
 - `puts()` - Write a string

Example:

```
int age;
float height;
char name[20];
// Taking input
printf("Enter your name: ");
scanf("%s", name);
printf("Enter your age: ");
scanf("%d", &age);
printf("Enter your height: ");
scanf("%f", &height);
// Displaying output
printf("\nName: %s", name);
printf("\nAge: %d", age);
printf("\nHeight: %.2f", height);
```

10. Functions in C

Functions are blocks of code that perform a specific task.

```
// Function definition
int add(int a, int b) {
    int sum = a + b;
    return sum;
}
// Function call
int main() {
    int result = add(5, 7); // Calling function
    printf("Sum = %d", result);
    return 0;
}
```

Key Points:

- Functions improve code readability and reusability.
- A function can take parameters and return a value.
- `main()` is the special function where program execution begins.
- `return` statement sends a value back to the caller.

11. Preprocessor Directives

Preprocessor directives start with `#` and are processed before compilation.

Directive	Meaning	Example
<code>#include</code>	Includes header files.	<code>#include <stdio.h></code>
<code>#define</code>	Defines a macro or constant.	<code>#define PI 3.14159</code>
<code>#undef</code>	Undefines a macro.	<code>#undef PI</code>
<code>#if #elif #else #endif</code>	Conditional compilation.	<code>#if X > 0</code> <code>...</code> <code>#endif</code>



Practice regularly and write more programs to become a better C programmer!

@coders.world

27. Arrays in C

An array stores multiple values of the same type in a contiguous memory location.

Syntax

```
dataType arrayName[size];
```

Example

```
int marks[5] = {85, 90, 78, 92, 88};  
// Accessing elements (0-based indexing)  
printf("First mark: %d\n", marks[0]);  
printf("Third mark: %d\n", marks[2]);
```

Key Points

- Indexing starts from 0.
- Array size must be a positive integer.
- All elements in an array are of the same type.
- Use `sizeof(array) / sizeof(array[0])` to find the size.
- Arrays are passed by reference to functions.

Example: Traversing an Array

```
int i;  
for(i = 0; i < 5; i++) {  
    printf("marks[%d] = %d\n", i, marks[i]);  
}
```

28. Pointers in C

A pointer is a variable that stores the memory address of another variable.

Syntax

```
dataType *pointerName;
```

Example

```
int num = 25;  
int *ptr; // Pointer declaration  
ptr = &num; // Address of num  
printf("Value: %d\n", num); // 25  
printf("Address: %p\n", (void*)&num);  
printf("Pointer Address: %p\n", (void*)ptr);  
printf("Value using Pointer: %d\n", *ptr); // 25
```

Pointer Operators

Operator	Meaning	Example
&	Address-of operator	&num → address of num
*	Dereference operator	*ptr → value at addr
->	Arrow operator	ptr->age → using pointer to structure



Remember:

Pointer = Address

*Pointer = Value at Address



29. Dynamic Memory Allocation

In C, memory can be allocated at runtime using `stdlib.h` functions.

Functions

Function	Purpose	Header File
<code>malloc()</code>	Allocates memory	<code>stdlib.h</code>
<code>calloc()</code>	Allocates & initializes memory to zero	<code>stdlib.h</code>
<code>realloc()</code>	Resizes previously allocated memory	<code>stdlib.h</code>
<code>free()</code>	Frees allocated memory	<code>stdlib.h</code>

Example

```
int *arr, n, i;  
printf("Enter size: ");  
scanf("%d", &n);  
arr = (int*)malloc(n * sizeof(int)); // allocate  
if(arr == NULL) {  
    printf("Memory allocation failed!\n");  
    return 1;  
}  
for(i = 0; i < n; i++)  
    arr[i] = i * 10;  
free(arr); // don't forget to free memory!
```



Tip: Always free the memory that you allocate.
Memory leaks can slow down your program. 😊



Happy Coding! Keep Practicing! ❤️

@coders.world

13. Strings in C

In C, strings are arrays of characters terminated by a null character '\0'.

Declaring and Initializing Strings

```
char str1[20];           // Declaration
char str2[] = "Hello";   // Auto size
char str3[20] = "World"; // With size
char str4[20] = {'H','i','\0'}; // Character list
```

Points to Remember

- Always leave space for the null character '\0'.
- String length = number of characters before '\0'.
- Arrays can be modified, string literals cannot.

Common String Functions (in <string.h>)

```
strlen(str) - Returns length of string
strcpy(dest, src) - Copies src to dest
strcat(dest, src) - Concatenates src to dest
strcmp(str1, str2) - Compares two strings
strrev(str) - Reverses the string
```

Example

```
char s1[20] = "Hello";
char s2[20] = " C World";
strcat(s1, s2);
printf("%s\n", s1);
// Output: Hello C World
```

14. Pointers in C

A pointer is a variable that stores the memory address of another variable.

Declaring and Using Pointers

```
int x = 10;
int *ptr;           // Pointer declaration
ptr = &x;           // Address of x
printf("Value of x: %d\n", x);           // 10
printf("Address of x: %p\n", (void*)&x); // Address
printf("Value at ptr: %d\n", *ptr);      // 10
*ptr = 20;           // Change value of x
printf("New value of x: %d\n", x);       // 20
```

Important Operators

```
& - Address of operator
* - Indirection (value at address)
ptr - Pointer variable
*ptr - Value stored at address pointed by ptr
```

15. Dynamic Memory Allocation

C allows dynamic memory allocation using functions from <stdlib.h>.

```
malloc(size) - Allocates size bytes of memory.
calloc(n, size) - Allocates memory for n elements of size bytes and initializes to 0.
realloc(ptr, size) - Resizes the memory block pointed by ptr.
free(ptr) - Deallocates the memory block.
```

Example

```
int *p;
p = (int *)malloc(5 * sizeof(int));
if (p == NULL) {
    printf("Memory not allocated");
}
free(p);
p = NULL;
```



Practice consistently and your C skills will keep improving!

@coders.world

16. Structures in C

Structures allow you to group different data types under a single name.

Defining a Structure

```
struct Student {  
    int roll;           // Integer  
    char name[20];      // String  
    float marks;        // Float  
};  
struct Student s1;     // Declaring variable
```

Example

```
struct Student s = {101, "Bob", 92.3}; // Initialization  
printf("%d %s %.2f\n", s.roll, s.name, s.marks);  
// Output: 101 Bob 92.30
```

Accessing Structure Members

```
s1.roll = 101;  
strcpy(s1.name, "Alice");  
s1.marks = 87.5;  
printf("Roll: %d\n", s1.roll);  
printf("Name: %s\n", s1.name);  
printf("Marks: %.2f\n", s1.marks);
```

Points to Remember

- Use . (dot) to access structure members.
- Structures can contain other structures.
- Arrays and pointers can be members of a structure.

17. Typedef in C

typedef is used to create an alias (new name) for an existing data type.

Syntax

```
typedef existingType newTypeName;
```

Example

```
typedef unsigned long int ulong;  
ulong a = 100000UL;  
printf("Value: %lu\n", a);  
// Output: Value: 100000
```

Common Use

- Makes complex types easier to use.
- Improves code readability.
- Useful while working with structures and pointers.

Example with Structure

```
typedef struct Student {  
    int roll;  
    char name[20];  
    float marks;  
} Student;  
Student s1 = {102, "Charlie", 88.6};
```

18. Preprocessor Directives

Preprocessor directives are instructions given to the compiler before compilation.

Directive	Purpose	Example
#include	Includes header files.	#include <stdio.h>
#define	Defines a macro or constant.	#define PI 3.14159
#undef	Undefines a macro.	#undef PI
#if, #else, #endif	Conditional compilation.	#if X > 0 ... #endif
#ifdef, #ifndef	Checks if a macro is defined or not.	#ifdef DEBUG ... #endif



Practice regularly, write more code, and build real projects!

@coders.world

23. Control Statements in C

Control statements allow you to control the flow of execution of a program.

23.1 if Statement

```
// Syntax
if (condition) {
    // code if condition is true
}

// Example
int x = 10;
if (x > 0) {
    printf("x is positive");
}
```

Notes:

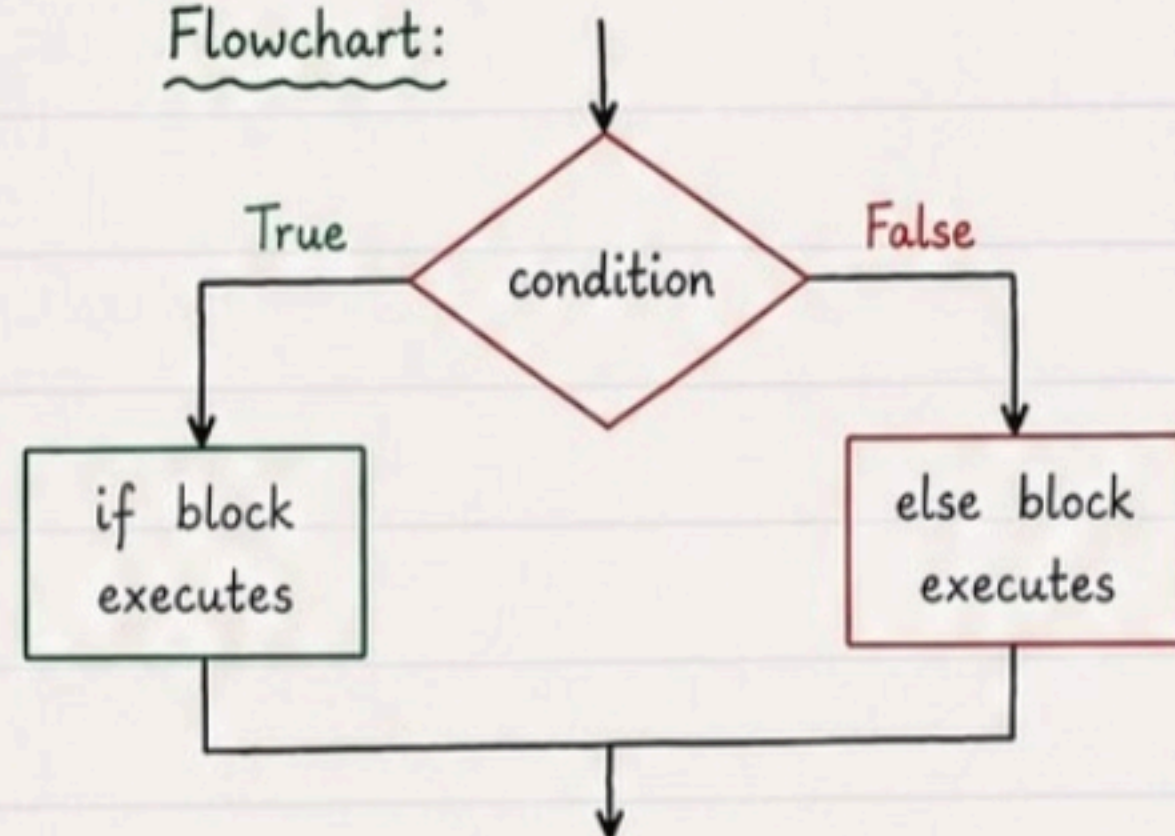
- condition must be a relational or logical expression.
- If condition is true, the block inside if is executed.
- If condition is false, the block inside if is skipped.

23.2 if-else Statement

```
// Syntax
if (condition) {
    // code if true
} else {
    // code if false
}

// Example
int num = -5;
if (num >= 0) {
    printf("Positive number");
} else {
    printf("Negative number");
}
```

Flowchart:



Tip:

Use meaningful variable names to make conditions easy to read. ★

23.3 else-if Ladder

```
// Syntax
if (condition1) {
    // code
} else if (condition2) {
    // code
} else if (condition3) {
    // code
} else {
    // code
}
```

Example:

```
int marks = 85;
if (marks >= 90) {
    printf("Grade A");
} else if (marks >= 75) {
    printf("Grade B");
} else if (marks >= 50) {
    printf("Grade C");
} else {
    printf("Fail");
}
```

Use When:

- Multiple conditions need to be checked.
- Only one block should execute.
- Conditions are mutually exclusive.



Control flow is the heart of any program - use it wisely!

12. Arrays in C

Arrays are used to store multiple values of the same type in a contiguous memory location.

12.1 Declaring and Initializing Arrays

```
// Declaration
int numbers[5];
float marks[10];

// Initialization
int arr[5] = {1, 2, 3, 4, 5};
char name[] = "C Language";
```

Notes:

- Array index starts from 0.
- Size of array must be a positive integer constant.
- Memory is allocated contiguously.

12.2 Accessing Array Elements

```
int arr[5] = {10, 20, 30, 40, 50};

printf("First element: %d\n", arr[0]); // 10
printf("Third element: %d\n", arr[2]); // 30
arr[4] = 100; // Update last element
```

Valid index range:

0 to (size - 1)

For arr[5] valid index is 0 to 4.

12.3 Array Example

```
#include <stdio.h>
int main() {
    int i, arr[5]; // Declaration
    // Input
    for (i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }
    // Output
    printf("\nArray elements:\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Example Run:

Input: 5 10 15 20 25

Output:

Array elements:

5 10 15 20 25

12.4 Multidimensional Arrays

Arrays with more than one dimension.

```
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d", matrix[1][2]); // Output: 6
```

- matrix[rows][columns]
- Access element using matrix[i][j]



Practice writing programs using arrays - it builds a strong foundation!

24. Functions in C

Functions break down a large program into smaller modules.

24.1 Function Declaration

```
// Syntax
returnType functionName(parameters);

// Example
int add(int a, int b); // Declaration
```

24.2 Function Definition

```
returnType functionName(parameters) {
    // code
    return value; // if returnType is not void
}
```

24.3 Function Call

```
// Syntax
functionName(arguments);

// Example
int sum = add(10, 20); // Function call
```

Notes:

- Function declaration tells the compiler about the function.
- Function definition contains the actual code.
- Function call transfers control to the function.
- Functions can return a value or be void.

25. Recursion in C

Recursion is when a function calls itself to solve a smaller instance of the problem.

Example: Factorial of a number

```
int factorial(int n) {
    if (n == 0)
        return 1; // Base case
    else
        return n * factorial(n - 1); // Recursive call
}
```

Caution:

- Always define a base case.
- Make sure the problem size reduces every time.
- Otherwise, it may cause **infinite recursion**. ⚠

How it Works:

- Base case stops the recursion.
- Each call reduces the problem size.
- Finally, results are combined while returning back.

Trace for factorial(4):

```
4 * factorial(3)
= 4 * 3 * factorial(2)
= 4 * 3 * 2 * factorial(1)
= 4 * 3 * 2 * 1
= 24
```

26. File Handling in C

File handling allows us to store data permanently.

26.1 Opening a File

```
FILE *fp;
fp = fopen("example.txt", "r"); // read mode
fp = fopen("example.txt", "w"); // write mode
fp = fopen("example.txt", "a"); // append mode
```

26.2 Common Functions

Function	Purpose
fopen()	Open a file
fclose()	Close a file
fprintf()	Write formatted data to a file
fscanf()	Read formatted data from a file
fgetc()	Read a character from a file
fputc()	Write a character to a file



Keep practicing small programs and you'll master C in no time! 😊

@coders.world

19. Common C Libraries

C provides a standard library of header files with useful functions.

Header File	Purpose	Common Functions
<stdio.h>	Input / Output operations	printf(), scanf(), getchar(), puts()
<stdlib.h>	General utilities	malloc(), free(), exit(), rand()
<math.h>	Mathematical functions	sqrt(), pow(), sin(), cos(), fabs()
<string.h>	String handling functions	strlen(), strcpy(), strcat(), strcmp()
<ctype.h>	Character handling functions	isalpha(), isdigit(), toupper(), tolower()
<time.h>	Time and date functions	time(), ctime(), localtime()

20. Storage Classes in C

Storage class defines the scope, lifetime and visibility of variables.

- **auto** : Default storage class for local variables.
- **register** : Suggests compiler to store variable in register (faster access).
- **static** : Retains value between function calls (for local variables) and limits scope to the block (for global variables).
- **extern** : Declares a global variable defined in another file.
- **typedef** : Creates an alias for an existing data type.

Example

```
typedef unsigned long int ulong;  
ulong count = 0;  
// count is now of type ulong
```

21. Bitwise Operators in C

Bitwise operators work on individual bits of integers.

Operator	Meaning	Example
&	Bitwise AND	5 & 3 → 1 (0101 & 0011 = 0001)
	Bitwise OR	5 3 → 7 (0101 0011 = 0111)
^	Bitwise XOR	5 ^ 3 → 6 (0101 ^ 0011 = 0110)
~	Bitwise NOT	~5 → -6 (Inverts all bits)
<<	Left shift	5 << 1 → 10 (0101 << 1 = 1010)
>>	Right shift	5 >> 1 → 2 (0101 >> 1 = 0010)

Example Program

```
int a = 5, b = 3;  
printf("AND : %d\n", a & b);  
printf("OR : %d\n", a | b);  
printf("XOR : %d\n", a ^ b);  
printf("NOT a: %d\n", ~a);  
printf("a<<1 : %d\n", a << 1);  
printf("a>>1 : %d\n", a >> 1);
```

22. Tips for C Programmers

- Always initialize variables before using them.
- Use meaningful variable and function names.
- Keep your code modular (use functions).
- Use comments to explain complex logic.
- Test your code with different inputs.
- Follow consistent indentation and formatting.



Remember:

Practice + Patience + Persistence
= Great Programmer 😊



Code. Compile. Debug. Repeat. That's the path to mastery!

@coders.world